

Laboratorio 2 - *Biblical Text Mining*

Curso: Programación Científica

Cristhian A. Rabi R.
cristhian.rabi@ce.ucn.cl

12 de mayo de 2026

1. Contexto

La Biblia es uno de los textos más influyentes y estudiados de la historia. Además de su relevancia histórica, cultural y literaria, constituye un corpus textual de gran tamaño y complejidad, compuesto por decenas de libros, múltiples autores, distintos estilos narrativos y miles de versículos.

En este laboratorio trabajaremos con la Biblia como fuente de datos, utilizando herramientas computacionales para explorar patrones lingüísticos, frecuencia de palabras, similitud entre textos, clasificación automática y representación vectorial del lenguaje.

La Biblia corresponde a una colección de libros escritos en distintos períodos históricos y atribuidos a múltiples autores. Su estructura se organiza jerárquicamente en libros, capítulos y versículos, permitiendo dividir el texto en unidades pequeñas de análisis. Tradicionalmente, la Biblia se divide en dos grandes secciones: Antiguo Testamento y Nuevo Testamento. El Antiguo Testamento contiene libros históricos, poéticos, legales y proféticos, mientras que el Nuevo Testamento incluye evangelios, cartas y textos narrativos relacionados con el cristianismo primitivo.

Cada libro posee características lingüísticas y estilísticas distintas. Algunos contienen narraciones históricas extensas, otros presentan poesía, leyes, genealogías, discursos o cartas. Esta diversidad convierte a la Biblia en un corpus especialmente interesante para el análisis computacional del lenguaje, ya que permite comparar estilos de escritura, vocabulario, estructuras sintácticas y distribución temática entre diferentes autores y períodos.

La Biblia posee una estructura jerárquica bien definida que permite organizar y analizar el texto en distintos niveles. Esta organización facilita su representación computacional y el procesamiento automatizado de información textual.

La jerarquía principal utilizada en este laboratorio será la siguiente:

```
Biblia
  Testamento
    Libro
      Capítulo
        Versículo
```

El versículo corresponde a la unidad textual más pequeña utilizada en el dataset y será considerado como una unidad básica de análisis. Desde una perspectiva lingüística y computacional, también es posible considerar unidades más pequeñas de análisis, como palabras individuales, subpalabras o incluso caracteres.

2. Objetivo

Desarrollar un sistema en Python orientado al análisis computacional de texto sobre el corpus bíblico, aplicando técnicas de procesamiento de lenguaje natural, representación vectorial, análisis estadístico y modelos probabilísticos para explorar patrones lingüísticos, similitud semántica y generación automática de texto.

2.1. Objetivos específicos

- Implementar un pipeline de preprocesamiento textual sobre un corpus.
- Representar versículos y libros mediante estructuras numéricas utilizando TF-IDF.
- Construir visualizaciones que permitan analizar patrones lingüísticos y relaciones semánticas dentro del corpus.
- Aplicar técnicas de reducción de dimensionalidad para visualizar datos textuales de alta dimensión.
- Desarrollar un motor de búsqueda semántico basado en similitud del coseno.
- Entrenar y evaluar modelos de clasificación de texto.
- Implementar modelos generativos probabilísticos basados en n-gramas.
- Analizar el comportamiento y limitaciones de modelos estadísticos aplicados al lenguaje natural.
- Interpretar resultados desde una perspectiva computacional, estadística y lingüística.

3. Requerimientos

3.1. Preprocesamiento de texto

El objetivo de esta etapa es transformar el texto original en una representación más adecuada para el análisis. El pipeline de preprocesamiento deberá incluir al menos las siguientes etapas:

- Conversión de texto a minúsculas.
- Eliminación de signos de puntuación.
- Eliminación de caracteres especiales y números.
- Tokenización de palabras.
- Eliminación de palabras vacías (*stopwords*).
- Construcción de vocabulario.
- Cálculo de frecuencias de palabras.

Adicionalmente, cada equipo deberá justificar las decisiones tomadas durante el preprocesamiento y analizar cómo estas afectan los resultados obtenidos. Puede añadir más etapas para enriquecer el trabajo.

3.2. Visualización y análisis exploratorio

Desarrollar visualizaciones que permitan analizar el comportamiento y estructura del corpus textual. El objetivo es representar gráficamente patrones presentes en los datos y apoyar la interpretación de los modelos desarrollados. Cada equipo deberá implementar al menos 3 visualizaciones distintas relacionadas con el análisis textual realizado. Algunas posibilidades incluyen:

- Distribución de longitud de versículos.
- Cantidad de versículos por libro.
- Matriz de similitud entre libros o capítulos.
- Evolución temporal de frecuencia de palabras a lo largo de los libros.
- Redes o grafos de co-ocurrencia entre palabras.
- Nubes de palabras.
- Comparación visual entre modelos unigram, bigram y trigram.

Se pide obligatoriamente un heatmap de similitud entre libros. Es una matriz de $N \times N$ donde N es la cantidad de libros.

3.3. Visualización de versículos utilizando PCA

Construir una visualización bidimensional de versículos utilizando reducción de dimensionalidad mediante PCA (*Principal Component Analysis*). Para ello, cada versículo deberá ser representado numéricamente mediante una matriz TF-IDF. Luego, se aplicará PCA para reducir dicha representación a dos dimensiones, permitiendo graficar cada versículo como un punto en un plano cartesiano. El objetivo de esta visualización es observar si existen agrupaciones naturales entre versículos, libros o testamentos a partir de su contenido textual. El gráfico deberá incluir:

- Cada punto debe representar un versículo.
- El eje x debe corresponder a la primera componente principal.
- El eje y debe corresponder a la segunda componente principal.
- Los puntos deben diferenciarse visualmente según libro, testamento u otra categoría definida por el equipo.
- El equipo deberá interpretar si se observan agrupaciones, separaciones o patrones relevantes.

Además, se deberá explicar brevemente qué representa PCA en este contexto y por qué es útil para visualizar datos textuales de alta dimensión.

3.4. Motor de búsqueda semántico

Dada una frase ingresada por el usuario, o un versículo del dataset, el sistema debe encontrar los K versículos más similares dentro del corpus. Para ello, se deberá vectorizar el texto utilizando TF-IDF y calcular la similitud entre documentos mediante similitud del coseno.

El resultado debe mostrar el ranking de versículos más similares, incluyendo libro, capítulo, versículo, texto original y valor de similitud.

3.5. Clasificador de versículos

Dado un versículo, el sistema debe predecir a qué libro de la Biblia pertenece. Para ello, se deberá entrenar un modelo de clasificación utilizando como entrada la representación textual del versículo y como etiqueta el libro correspondiente. El equipo deberá separar los datos en entrenamiento y prueba, entrenar al menos un clasificador y reportar métricas de evaluación como accuracy y matriz de confusión.

3.6. Modelo generador de texto

Desarrollar un generador de *versículos falsos* utilizando relaciones probabilísticas entre palabras. El objetivo es construir modelos de lenguaje estadísticos basados en frecuencias y probabilidades de aparición de secuencias de palabras, sin utilizar modelos de lenguaje preentrenados ni redes neuronales.

- Generar secuencias de al menos 15 palabras.
- Permitir que la generación comience desde una palabra inicial elegida desde algún versículo real o ingresada por el usuario.
- Incorporar tokens especiales de inicio y término, por ejemplo <START> y <END>.
- Detener la generación al alcanzar una longitud máxima o al llegar al token final <END>.
- Implementar y comparar cuatro modelos: unigram, bigram, trigram y un modelo n-gram donde n sea definido por el equipo.
- Comparar cualitativamente la coherencia de los textos generados por cada modelo.
- Analizar las limitaciones observadas, tales como repetición, falta de coherencia, secuencias sin salida o alta dependencia del corpus original.

3.7. Análisis de sentimiento a lo largo de los capítulos

Realizar un análisis de sentimiento sobre el corpus bíblico con el objetivo de estudiar cómo varía el tono emocional entre distintos capítulos o libros. Para ello, se deberá asignar un puntaje de sentimiento a cada versículo utilizando alguna técnica de análisis de sentimiento disponible en Python. Posteriormente, los puntajes deberán agregarse por capítulo o libro para obtener una representación global del sentimiento a lo largo del texto. El análisis deberá incluir:

- Cálculo de puntajes de sentimiento para cada versículo.
- Agregación de resultados por capítulo o libro.
- Visualización de la evolución del sentimiento mediante gráficos.
- Identificación de capítulos o libros con sentimientos extremos.
- Interpretación de los patrones observados.

El equipo deberá además discutir las limitaciones del análisis de sentimiento aplicado a textos antiguos o religiosos, considerando posibles problemas de vocabulario, traducción, contexto y ambigüedad semántica.

3.8. Análisis e interpretación de resultados

Realizar un análisis crítico de los resultados obtenidos en las distintas etapas del laboratorio. No basta únicamente con generar gráficos o ejecutar modelos; se espera interpretar el comportamiento observado y relacionarlo con las características del corpus textual. El análisis deberá incluir al menos:

- Interpretación de patrones observados en las visualizaciones.
- Comparación entre distintos modelos y técnicas utilizadas.
- Discusión sobre las limitaciones de TF-IDF y modelos basados en n-gramas.
- Identificación de posibles sesgos o problemas presentes en el corpus.
- Impacto del preprocesamiento sobre los resultados finales.
- Explicación de casos exitosos y casos fallidos.

Se valorará especialmente la capacidad de interpretar resultados desde una perspectiva estadística, lingüística y computacional.

4. Consideraciones

- TF-IDF debe implementarlo el equipo, no puede utilizar librerías que lo implementen
- Utilizar programación orientada a objetos
- Usar Pandas, aunque puede haber excepciones en algunos ítems
- Para grafos puede utilizar NetworkX
- Para PCA puede utilizar scikit-learn
- El cálculo de similitud del coseno debe ser implementado por el equipo
- Dataset: <https://www.kaggle.com/datasets/oswinrh/bible>, puede seleccionar cualquier versión, o buscar alguna en español si lo desea.

5. Entregables

- Código en un repositorio Github, con un README pertinente, incluir diagrama de clases
- Informe con los resultados y conclusiones

6. Interrogación

Una vez entregado el Taller, todos los equipos pueden ser interrogados. De ser así, el puntaje del taller se ponderará con el porcentaje de logro obtenido en la interrogación, pudiendo así reprobar la entrega a pesar de haber tenido un 100 % de logro por ejemplo. La nota es por equipo, no hay nota individual.

7. Orientaciones

Se espera que cada equipo lo desarrolle con su *toque* propio. Lo más importante en este Taller es el análisis de los resultados. Se espera también que cada equipo vaya mostrando al docente su avance al menos una vez, será bien valorado.

8. Entrega

- Se entrega por Campus Virtual, el repositorio puede ir en un enlace dentro de un .txt.
- Fecha límite: 29 de junio, 23:59